LITE THINKING INSTITUTE OF TECHNOLOGY

CERTIFICACIÓN 6 · SESIÓN 6

Reto 6 — Pipeline Multiagente
Guía de cumplimiento paso a paso

Cómo se cumple cada entregable y cómo reproducir las evidencias

Caso 10 · Clínica Vitalis — anonimización → clasificación → escritura

Proyecto: `Reto Practico_Pipeline_Multiagente`

> **Estado.** Los 4 entregables están generados y verificados. El pipeline corrió sobre 3 historias propias y las 12 del dataset de clase, con 28 pruebas automáticas en verde, 19 de 19 casos de prueba aprobados y verificación anti-fuga limpia sobre los archivos releídos desde disco.
>
> **Pendiente opcional:** correr la Etapa 2 contra un modelo real vía OpenRouter (Track A), que exige una API key propia. Ver la sección 7.

1. Qué pide el reto

Demostrar, **de forma individual y con datos propios**, que el pipeline multiagente funciona y que usted puede verificarlo — no solo construirlo.

#	Entregable	Criterio de calificación	Peso
1	Crear 3 historias clínicas sintéticas nuevas, inventadas, con los 4 identificadores (nombre, cédula, dirección, teléfono) mezclados en texto libre.	Las 3 son propias, con los 4 identificadores presentes y una condición médica clara y distinta entre sí.	20%
2	Correr el pipeline (Track A o Track B) sobre esas 3 historias.	El `Formulario_Salida` generado no contiene ningún dato identificable.	30%
3	Completar al menos 5 filas de `Historias_de_Prueba_Pipeline_Clinico.xlsx` con resultados reales (Resultado obtenido y Estado).	Al menos 5 casos con evidencia real, no solo «Pasó» sin justificación.	30%
4	Responder en máximo 150 palabras: ¿qué fue automatización pura y qué requirió criterio del agente? Con un ejemplo de sus 3 historias.	Conecta el marco automatización vs. agente con un ejemplo propio.	20%

> **Dónde está la nota.** El 60% (puntos 2 y 3) no evalúa que el pipeline corra, sino que se *demuestre* que corrió bien: que la salida no filtra datos y que los casos de prueba traen evidencia concreta.

Track A o Track B

	Track A	Track B
Qué es	El Agente 2 llama a un modelo real por API.	El Agente 2 se resuelve turno por turno en el chat.
API key	Sí (OpenRouter).	No.
Costo	Consumo por token.	Ninguno.

> **Ruta elegida:** los Agentes 1 y 3 son código ejecutado (son deterministas: no ganan nada pasando por un LLM) y el Agente 2 se resolvió por **Track B**. El código de Track A está construido y listo para usarse con una key propia. Ambos tracks reciben exactamente el mismo contrato de entrada.

2. Estado de cada entregable

#	Entregable	Archivo	Estado
1	3 historias propias	`data/output/Historias_Propias_Reto.xlsx`	**Completo**
2	Pipeline corrido	`data/output/Formulario_Salida_Historias_Propias.xlsx`	**Completo**
3	Historias de prueba	`data/output/Historias_de_Prueba_COMPLETADO.xlsx`	**19 de 19**
4	Reflexión escrita	`evidencias/reflexion_escrita.md`	**142 palabras**
+	Corrida sobre las 12 de clase	`data/output/Formulario_Salida_Dataset_Original.xlsx`	**Extra**
+	Suite de pruebas	`evidencias/salida_pruebas.txt`	**28 en verde**
+	Contraste con el oráculo	`evidencias/comparacion_con_oraculo.md`	**Extra**
+	Dossier de evidencias	`evidencias/Dossier_Evidencias_Reto6.pdf`	**Extra**

Resultado de la corrida sobre las historias propias

```
$ python3 orquestador.py --fuente propias --track b

  PIPELINE MULTIAGENTE — CLINICA VITALIS
  Fuente: Historias_Propias (3 registros)   ·   Track: B

ETAPA 1 · Anonimizacion (deterministica, sin LLM, sin red)
  Anonimizados sin residuos : 3/3
  Mapa de auditoria         : data/internal/mapa_auditoria_Historias_Propias.xlsx

ETAPA 2 · Clasificacion clinica (Track B)
  Clasificados              : 3/3
  Marcados 'indeterminado'  : 0

ETAPA 3 · Escritura y verificacion anti-fuga final
  Archivo                   : data/output/Formulario_Salida_Historias_Propias.xlsx
  Filas escritas            : 3
  Celdas escaneadas         : 32
  Verificacion anti-fuga    : LIMPIO
```

Una decisión técnica

El documento de arquitectura del curso propone `openpyxl`, `pandas`, `pydantic` y `pytest`. En el equipo de desarrollo no había `pip`, así que se usaron equivalentes de la librería estándar. Ningún requisito de fondo cambia:

Propuesto	Usado aquí	Requisito que se mantiene
`openpyxl` + `pandas`	`zipfile` + `xml.etree`	Leer y escribir .xlsx respetando encabezados y orden
`pydantic`	`dataclasses` con validación	Campos explícitos, nunca un diccionario abierto
`httpx` / `requests`	`urllib.request`	Reintentos con espera creciente, sin registrar la API key
`pytest`	`unittest`	Las pruebas obligatorias de la sección 10.2

3. Entregable 1 — Las 3 historias propias (20%)

El criterio exige tres cosas a la vez: que sean **propias**, que tengan los **4 identificadores** mezclados en el texto libre, y que cada una tenga una **condición médica clara y distinta** de las otras dos.

Paso 1 · Tres condiciones distintas entre sí

Registro	Especialidad	Condición	Evidencia en la nota
`REG-P01`	Cardiovascular	Hipertensión arterial	Presión arterial 158/96 mmHg (una sola toma), cefalea y mareos de 2 semanas
`REG-P02`	Urología	Infección urinaria baja	Leucocitos 25-30 por campo y nitritos positivos, con ardor al orinar de 3 días
`REG-P03`	Endocrinología	Diabetes mellitus tipo 2	Glucemia en ayunas 186 mg/dL y HbA1c 8,2 %, con sed y poliuria de 1 mes

> **A tener en cuenta.** El dataset de clase también incluye casos de hipertensión y diabetes. El criterio pide que las 3 condiciones sean distintas *entre sí*, y lo son; las historias, los valores clínicos y los identificadores son propios. Si en la sustentación se pregunta, la infección urinaria (REG-P02) es la condición que no aparece en el material de clase.

Paso 2 · Los 4 identificadores dentro de la nota

No basta con tenerlos en las columnas: el ejercicio consiste en que el Agente 1 los encuentre *dentro del texto libre*. Así quedó `REG-P01` (identificadores en negrita):

> Paciente **Carlos Andrés Mendoza Rojas**, CC **12.345.678**, 45 años, vive en **Cl 10 #5-20, Santa Cruz de la Sierra**, tel. **315-555-0123**. Consulta por dolor de cabeza ocasional, mareos y sensación de presión en la cabeza desde hace aproximadamente 2 semanas. Presión arterial 158/96 mmHg, frecuencia cardíaca 82 lpm. Padre con antecedente de hipertensión arterial. Fuma aproximadamente 5 cigarrillos al día. Consume alcohol ocasionalmente, unas 2 veces al mes.

Todos los datos personales son ficticios. Los teléfonos usan el rango `555`, reservado para ficción, igual que el dataset de clase.

> **Verificación automática.** La prueba `test_los_4_identificadores_estan_presentes_en_la_entrada` falla si a alguna historia le falta un identificador o si alguno no aparece literalmente en la nota.

4. Entregable 2 — Correr el pipeline (30%)

Lo que se califica no es que el pipeline corra, sino que el `Formulario_Salida` **no contenga ningún dato identificable**. Por eso la garantía está en el contrato entre etapas, no en la disciplina de quien lo opera.

```
$ python3 orquestador.py --fuente propias --track b
```

Etapa 1 · Agente 1 — Anonimización (determinístico, sin LLM, sin red)

Asigna el código `PAC-NNN` y arma la lista de cadenas a borrar **a partir de las columnas de la propia fila**, nunca inventadas. Para cada identificador genera sus variantes de escritura. Con los datos de REG-P01:

- **Cédula** `12.345.678` → con puntos, sin puntos, con comas, con espacios y con prefijo `CC`.
- **Teléfono** `315-555-0123` → con guiones, espacios, puntos o corrido.
- **Nombre** → completo, cada componente de 3 o más letras y los pares consecutivos.
- **Dirección** → completa y la vía sin la ciudad.

Cada coincidencia se reemplaza por `[ANON]` y luego corre la verificación de dos capas. Si algo sobrevive, **el registro se detiene**: no se arregla en silencio ni pasa con una advertencia.

Etapa 2 · Agente 2 — Clasificación (el único que usa IA)

Recibe exclusivamente dos claves. Este es el paquete real que recibió para REG-P01:

```
{
  "codigo_paciente": "PAC-001",
  "texto_anonimizado": "Paciente [ANON] 45 años, vive en [ANON], tel. [ANON]. Consulta por dolor de cabeza ocasional,
mareos y sensación de presión en la cabeza desde hace aproximadamente 2 semanas. Presión arterial 158/96 mmHg,
frecuencia cardíaca 82 lpm. Padre con antecedente de hipertensión arterial. Fuma aproximadamente 5 cigarrillos al día.
Consume alcohol ocasionalmente, unas 2 veces al mes."
}
```

Cualquier clave adicional lanza `ViolacionDeContrato` antes de salir a la red. El clasificador no «se acuerda» de no filtrar: **no puede filtrar lo que nunca tuvo**.

Etapa 3 · Agente 3 — Escritura (determinístico + última línea de defensa)

Escribe el .xlsx con los 8 encabezados en el orden exacto del archivo de referencia, sin la fila de ejemplo y sin la hoja de entrada. Después **relee el archivo desde disco** y lo escanea contra los identificadores originales: lo que importa es lo que quedó escrito.

Resultado obtenido

Código	Enfermedad probable	Riesgo	Confianza	Fuma	Antecedentes
PAC-001	hipertensión arterial estadio 2 (por confirmar)	medio	media	Sí	padre con hipertensión arterial
PAC-002	infección urinaria baja (cistitis)	bajo	alta	No	infección urinaria previa hace aproximadamente 1 año
PAC-003	diabetes mellitus tipo 2	medio	alta	No	madre con diabetes mellitus tipo 2

Criterio del 30% cumplido. El escaneo recorrió las 32 celdas del archivo final y no encontró ninguna de las 12 cadenas identificadoras (3 historias × 4), ni exacta, ni con puntuación alterada, ni como fragmento suelto del nombre.

5. Entregable 3 — Las historias de prueba (30%)

El reto pide **al menos 5** casos con evidencia real. Se completaron **19**: los 14 de la plantilla y 5 nuevos sobre las historias propias. La evidencia se genera leyendo los archivos que produjo el pipeline; no está escrita a mano.

```
$ python3 generar_evidencias.py

Casos completados con evidencia real : 19
  Pasó   : 19
  Falló  : 0

Archivo -> data/output/Historias_de_Prueba_COMPLETADO.xlsx
```

Ejemplos de la evidencia producida

Caso	Resultado obtenido (texto real del archivo)	Estado
`TC-01`	Las 4 cadenas de REG-01 quedaron ausentes del texto: 'Marcela Suárez Rincón', '52.884.117', 'Cra 45 #12-30, Bogotá', '300-555-0111'. Se asignó el código PAC-001 y el texto quedó con 3 marcadores [ANON] (los contiguos se compactan en uno). Texto resultante: "Paciente [ANON] reside en [ANON], tel. [ANON], 54 años. Consulta por fatiga persistente y sed excesiva de 3 meses de evolución. Glucemia en ayunas 182..."	Pasó
`TC-02`	El payload serializado hacia el Agente 2 contiene exactamente 2 claves: ['codigo_paciente', 'texto_anonimizado']. validar_payload_agente2() se ejecuta como gate en el orquestador y lanza ViolacionDeContrato ante cualquier clave extra (prueba test_rechaza_clave_extra, en verde).	Pasó
`TC-04`	PAC-006 (REG-06) quedó como 'indeterminado' con confianza 'baja'. comportamiento_observado enumera la evidencia faltante: reactantes de fase aguda, factor reumatoide, anti-CCP e imagen. No se afirmó artritis reumatoide sin respaldo serológico. Extracto: "La rigidez matutina de más de una hora con compromiso de rodillas y manos orienta a artritis inflamatoria, pero la evidencia es in..."	Pasó
`TC-15`	3 de 3 historias anonimizadas, 0 detenidas. Se buscaron 12 cadenas identificadoras y sobrevivieron 0. Ejemplo REG-P01 → "Paciente [ANON] 45 años, vive en [ANON], tel. [ANON]. Consulta por dolor de cabeza ocasional, mareos y sensación de presión en la cabeza des..."	Pasó
`TC-16`	Formulario_Salida_Historias_Propias.xlsx releído desde disco: 3 filas, 32 celdas escaneadas contra los 12 identificadores (exactos, con puntuación alterada y por componente del nombre). Hallazgos: 0.	Pasó
`TC-19`	PAC-001 → 'hipertensión arterial estadio 2 (por confirmar)', confianza 'media'; la justificación explica la limitación: "La confianza es media y no alta porque hay una sola toma registrada: el diagnóstico exige confirmar cifras elevadas en mediciones repetidas. Riesgo me..." PAC-003 → 'diabetes mellitus tipo 2', confianza 'alta', sustentada en glucemia 186 mg/dL y HbA1c 8,2 %.	Pasó

La suite de pruebas que respalda las evidencias

```
$ python3 -m unittest discover -s tests
Ran 28 tests
OK
```

Grupo de pruebas	Qué verifica
`TestAntifugaAgente1`	Ninguna de las cadenas identificadoras sobrevive al Agente 1.
`TestContratoAgente2`	El payload hacia el LLM lleva exactamente 2 claves.
`TestReglaIndeterminado`	enfermedad_probable='indeterminado' exige confianza='baja'.
`TestAntifugaArchivoFinal`	El .xlsx entregable, releido desde disco, no contiene identificadores.
`TestAislamientoMapaAuditoria`	Ni el Agente 2 ni el Agente 3 tocan el mapa codigo <-> identidad.
`TestSeguridadCredenciales`	TC-13: la API key no aparece en texto plano en ningun archivo.
`TestCasosLimite`	Robustez: nota vacia y fallo de clasificacion.
`TestCoberturaCompleta`	TC-10 y TC-11: unicidad de codigos y cero historias omitidas.
`TestEntregablesDelReto`	Los 4 entregables existen tras una reconstruccion desde cero.

6. Entregable 4 — La reflexión escrita (20%)

El criterio pide conectar el marco de automatización vs. agente con un **ejemplo concreto de las propias historias**. Los dos errores que hacen perder este 20% son quedarse en la teoría sin ejemplo y pasarse de 150 palabras.

El razonamiento

Etapa	Naturaleza	Por qué
Agente 1	Automatización pura	La misma regla de sustitución para cualquier paciente, sin interpretar nada. Se verifica al 100%.
Agente 2	**Agente**	Texto libre con información incompleta: hay que interpretar y ponderar varios datos. Justifica el uso de un LLM.
Agente 3	Automatización pura	Mapeo fijo de columnas. Mismo resultado cada vez.

La respuesta entregada — 142 palabras

> **Automatización pura.** Las etapas de anonimización y escritura fueron automáticas. En REG-P02 se borraron nombre, cédula, dirección y teléfono. No hizo falta criterio porque la regla es siempre la misma para cualquier paciente: identifica los campos personales y los elimina, sin interpretar nada del caso.
>
> **Criterio.** La clasificación sí lo necesitó. En REG-P01 la presión era 158/96, pero el agente no dijo "hipertensión confirmada" porque había una sola medición: puede estar elevada por estrés o "efecto bata blanca", y el diagnóstico exige lecturas elevadas repetidas. Además, para decidir el nivel de riesgo combinó el antecedente del padre, el hábito de fumar y los demás datos clínicos.
>
> **Conclusión.** Una regla sirve cuando la tarea es repetitiva y predecible, con la misma acción para cualquier caso; hace falta un agente cuando hay que interpretar información incompleta, combinar varios datos y ponderarlos según el contexto.

Por qué el ejemplo funciona. No dice «el agente usó criterio» en abstracto: muestra que con una presión de 158/96 el agente no afirmó «hipertensión confirmada» porque había una sola medición, y que para el riesgo combinó el antecedente familiar y el tabaquismo. La clasificación real del Agente 2 lo confirma: confianza «media» por una sola toma registrada.

7. Qué falta (opcional)

Los 4 entregables están cumplidos. Lo que sigue fortalece la entrega, pero el reto no lo exige.

7.1 · Correr la Etapa 2 contra un modelo real (Track A)

> **Antes de nada.** No use la API key del material de clase: el PDF `Dia4/paso a paso n8n rack.pdf` trae una key de OpenRouter del instructor en texto plano. Debe considerarse comprometida.

1. Cree una cuenta y una key propia en `openrouter.ai`. Hay modelos con sufijo `:free` que sirven sin costo.

2. Guárdela fuera del código (el archivo `.env` ya está en `.gitignore`; es lo que verifica TC-13):

```
$ echo "OPENROUTER_API_KEY=su_clave_aqui" > .env
$ echo "OPENROUTER_MODEL=openai/gpt-4o-mini" >> .env
```

3. Corra y regenere las evidencias:

```
$ python3 orquestador.py --fuente propias --track a
$ python3 generar_evidencias.py
$ python3 -m unittest discover -s tests -v
```

Si alguna clasificación difiere de la de Track B, esa diferencia es material útil para la sustentación: muestra dónde el juicio clínico es genuinamente ambiguo.

7.2 · Mejoras que suman en la sustentación

Mejora	Qué aporta	Esfuerzo
Capturas de pantalla	La terminal con las pruebas en verde y el Excel final abierto.	Bajo
Repositorio en Git	Demuestra que `.env` y `data/internal/` nunca entraron al control de versiones. Refuerza TC-13.	Bajo
Interfaz web con Flask	Un botón «Correr pipeline» que muestre las 3 etapas (Paso 11 de la Sesión 6).	Medio

7.3 · Dos inconsistencias del material de clase

Al contrastar la corrida con el oráculo, el `Formulario_Salida` de referencia contradice a las Historias de Prueba del mismo paquete:

Caso	Dice la referencia	Exige la Historia de Prueba	Qué se hizo
`PAC-005`	`nivel_riesgo = medio`	TC-12: `nivel_riesgo = 'alto'` por el antecedente cardiovascular	Se siguió TC-12.
`PAC-006`	artritis reumatoide, confianza media	TC-04: no afirmar un diagnóstico sin respaldo	Se declaró `indeterminado` con confianza baja.

El detalle, junto con las demás divergencias de criterio clínico, está en `evidencias/comparacion_con_oraculo.md`.

8. Cómo regenerar todo desde cero

Desde la raíz del proyecto y en este orden (cada paso usa lo que genera el anterior). En Windows use `python` o `py` en lugar de `python3`.

```
# 1-2. Pipeline sobre las dos fuentes
python3 orquestador.py --fuente propias --track b
python3 orquestador.py --fuente dataset --track b

# 3. Historias de prueba con evidencia real
python3 generar_evidencias.py

# 4. Pruebas automáticas (la salida queda como evidencia)
python3 -m unittest discover -s tests 2> evidencias/salida_pruebas.txt

# 5. Contraste con el oráculo
python3 comparar_con_oraculo.py

# 6. Documentos PDF
python3 generar_dossier_evidencias.py
python3 generar_guia.py
```

Comando	Genera
`orquestador.py --fuente propias`	`Historias_Propias_Reto.xlsx`, `Formulario_Salida_Historias_Propias.xlsx` y el mapa de auditoría interno
`orquestador.py --fuente dataset`	`Formulario_Salida_Dataset_Original.xlsx` y el mapa de auditoría interno
`generar_evidencias.py`	`Historias_de_Prueba_COMPLETADO.xlsx` con 19 casos y `evidencias/aislamiento_secretos_git.txt`
`unittest discover`	`evidencias/salida_pruebas.txt` — 28 pruebas
`comparar_con_oraculo.py`	`evidencias/salida_comparacion_oraculo.txt`
`generar_dossier_evidencias.py`	`evidencias/Dossier_Evidencias_Reto6.pdf`
`generar_guia.py`	`Guia_Reto6_Pipeline_Multiagente.pdf` (este documento)

> **Qué NO se entrega nunca.** La carpeta `data/internal/` contiene el mapa `código_paciente ↔ identidad real`. Si se filtra, revierte toda la anonimización. Está en `.gitignore`, ningún agente posterior lo lee, y **no debe incluirse en lo que suba a la plataforma**.

9. Checklist de entrega

	Punto	Archivo	Estado
☑	3 historias propias, 4 identificadores, condiciones distintas	`Historias_Propias_Reto.xlsx`	Listo
☑	Pipeline corrido, salida sin datos identificables	`Formulario_Salida_Historias_Propias.xlsx`	Listo
☑	Mínimo 5 casos de prueba con evidencia real	`Historias_de_Prueba_COMPLETADO.xlsx` (19)	Listo
☑	Reflexión de máximo 150 palabras	`reflexion_escrita.md` (142)	Listo
☑	Código fuente del pipeline	`agents/` , `core/` , `tests/`	Listo
☑	Salida de las pruebas	`evidencias/salida_pruebas.txt`	Listo
☐	Corrida con modelo real vía OpenRouter	Requiere API key propia	Opcional
☐	Capturas de pantalla de la corrida	—	Opcional

Verificación final antes de subir

```
$ python3 -m unittest discover -s tests
Ran 28 tests
OK
```

Si esas 28 pruebas están en verde, entonces a la vez: las 3 historias tienen sus 4 identificadores, ninguna cadena identificadora sobrevive a la anonimización, el paquete hacia el clasificador lleva exactamente 2 claves, todo `indeterminado` tiene confianza baja, los archivos finales no filtran datos, el mapa de auditoría está aislado, no hay API keys en el código, los 4 entregables existen y el pipeline aguanta una historia corrupta sin caerse.

Estructura del proyecto

```
.
├── orquestador.py                  punto de entrada: corre las 3 etapas en orden
├── config.py                       rutas, nombres de hojas y configuración de OpenRouter
├── generar_evidencias.py           completa las Historias de Prueba (entregable 3)
├── comparar_con_oraculo.py         compara la corrida con el formulario de referencia
├── generar_dossier_evidencias.py   arma el PDF de evidencias
├── generar_guia.py                 arma esta guía
├── agents/                         los 3 agentes
├── core/                           contratos, reglas, anti-fuga, auditoría, cliente HTTP, E/S Excel
├── data/
│   ├── historias_propias.py        las 3 historias inventadas para el reto
│   ├── clasificaciones_track_b_*.json
│   ├── input/                      archivos de clase (solo lectura)
│   ├── internal/                   ← NO ENTREGAR · mapa código ⇔ identidad
│   └── output/                     ← los .xlsx de los entregables 1, 2 y 3
├── evidencias/                     reflexión (entregable 4), salidas de pruebas y dossier PDF
├── tests/test_pipeline.py          28 pruebas automáticas
└── README.md · .gitignore · Guia_Reto6_Pipeline_Multiagente.pdf
```